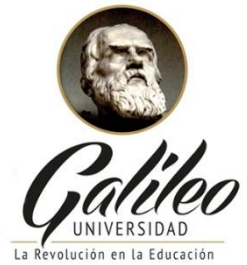


Universidad Galileo	Fecha: 14/09/2026
Facultad: FISSICC	Alumno: Angel Camargo
Curso: Microprocesadores	Carné: 24003664
Sección: A	Hora de Laboratorio: 1:00 pm
Auxiliar: Brandon Eduardo	Día de Laboratorio: Lunes



Laboratorio #4 Calling Assembly Routines from C

Objetivos:

Aprender el correcto procedimiento para poder desarrollar una aplicación la cual nos permita poder utilizar código escrito en los lenguajes de programación en C y ensamblador, comprendiendo como podemos hacer que una función la cual fue desarrollada en ensamblador puede ser llamada desde un programa en C. Además de igual manera buscamos comprender el uso de los registros del procesador ARM Cortex-M4 para el paso de los argumentos y el retorno de las funciones, igualmente analizar el funcionamiento de las rutinas dadas: `my_strcpy` y `my_capitalize` mediante el uso del depurador de Visual Studio Code.

Resumen:

En esta práctica desarrollamos una aplicación para el microcontrolador ARM Cortex-M4 utilizando los lenguajes de programación C y aparte de este se utilizó ensamblador. El programa utiliza dos rutinas las cuales tienen por nombre `my_strcpy` y `my_capitalize` las cuales fueron hechas en ensamblador. La primera permite poder copiar una cadena de caracteres desde una dirección de memoria hacia otra, y la segunda permite recorrer una cadena de caracteres y convierte las letras minúsculas a mayúsculas utilizando los códigos ASCII, finalmente utilizamos el depurador para poder observar los registros SP, LR Y PC, además de los contenidos de memoria y el comportamiento de las instrucciones en ensamblador.

Teoría:

En los sistemas embebidos normalmente se desarrollan en lenguaje C, debido a que este permite desarrollar software de manera rápida y modular. Pero el lenguaje ensamblador puede utilizarse para aquellas partes donde es necesario obtener un mejor rendimiento.

Combinar ambos lenguajes permite aprovechar las ventajas de cada uno de los dos lenguajes, como puede ser la facilidad y rapidez del desarrollo de C y el rendimiento y control de bajo nivel que proporciona el ensamblador. Para esta práctica utilizamos la combinación de estos dos para poder llamar rutinas escritas por medio de ensamblador desde un programa desarrollado en C.

Convenio de Registros

Los primeros cuatro argumentos de una función se tienen que pasar por medio de los registros R0, R1, R2 y R3, donde comenzamos primeramente por R0. Si existen más de cuatro, estos pueden ser enviado por medio del stack.

Los registros R0-R3 también se pueden utilizar como almacenamiento temporal cuando los argumentos ya no son necesarios. Los registros R4-R11 deben poder conservar su valor durante la ejecución de una subrutina. Si dicha subrutina necesita utilizarlos primeramente debe guardarlos y después restaurarlos, generalmente utilizando el stack. Por otra parte, el registro LR contiene una dirección de retorno de una subrutina. Cuando se ejecuta una instrucción de llamada. La dirección de retorno se almacena en LR.

El BX LR permite poder regresar a la dirección almacenar previamente en el Link Register. El PC contiene la dirección de la instrucción que se está ejecutando o de la siguiente instrucción según el contexto que tenga el procesador.

Directivas Utilizadas

- **AREA:** Define una sección del programa y permite especificar características de esa sección, como que contiene código ejecutable.
- **CODE:** Indica que la sección contiene código
- **READONLY:** Indica que la sección es de solo lectura
- **ALIGN=2:** Establece la alineación de la sección.
- **THUMB:** Indica que se utilizarán instrucciones del conjunto Thumb
- **REQUIRE8:** Indica que el código requiere que el tack este alineados a 8 bytes
- **PRESERVE8:** Indica que el código mantiene la alineación de 8 bytes del stack
- **EXPORT:** Hace que un símbolo o rutina pueda ser utilizado desde otros archivos

Datos Prácticos:

Pregunta 13

```
SP 0x20040000
SP_MAIN 0x20040000
SP_PROCESS 0x00000000
LR 0x00000479
PC 0x00000518
```

SP = 0x20040000

LR = 0x00000479

PC = 0x00000518

Pregunta 14

0x506	F2C20100	MOVT	r1, #0x2000
0x50A	F6470040	MOV	r0, #0x7840
0x50E	F2C0107D	MOVT	r0, #0x17d
0x512	6008	STR	r0, [r1, #0]
0x514	4770	BX	lr
0x516	0000	MOVS	r0, r0
0x518	B580	PUSH	{r7, lr}
0x51A	B082	SUB	sp, sp, #8
0x51C	2000	MOVS	r0, #0
0x51E	9001	STR	r0, [sp, #4]
0x520	F240505A	MOV	r0, #0x55a
0x524	F2C00000	MOVT	r0, #0
0x528	F2400104	MOVT	r1, #4

La flecha amarilla apunta a la instrucción PUSH {r7, lr}, con la dirección 0x518. La dirección corresponde al valor del registro PC (R15), que en dicho momento contiene 0x518

Pregunta 15

SP 0x2003FFF0
SP_MAIN 0x2003FFF0
SP_PROCESS 0x00000000
LR 0x00000479
PC 0x00000520

Como se puede ver los dos registros que cambiaron fueron el SP y el PC. La instrucción se relaciona con el PC porque después de ejecutar PUSH avanza a la dirección de la siguiente instrucción y el SP porque guarda los registros R7 y LR en la pila por lo que este disminuye 8 bytes para reservar espacio.

Pregunta 16

0x51E	9001	STR	r0, [sp, #4]
0x520	F240505A	MOV	r0, #0x55a
0x524	F2C00000	MOVT	r0, #0

Las dos instrucciones con 4 bytes serían r0, #0x55a y r0, #0

Pregunta 17

0x530	9100	STR	r1, [sp, #0]
0x532	F7FFFFAB	BL	my_strcpy ; 0x48C
0x536	0000	LDR	r0, [sp, #0]

SP 0x2003FFF0
SP_MAIN 0x2003FFF0
SP_PROCESS 0x00000000
LR 0x00000479
PC 0x00000532

SP = 0x2003FFF0, LR = 0x00000479, PC = 0x00000532

Pregunta 18

```
SP 0x2003FFF0
SP_MAIN 0x2003FFF0
SP_PROCESS 0x00000000
LR 0x00000537
PC 0x0000048C
```

0x488	F000F81C	BL	__seminhosting_library_function ; 0x4C4
0x48C	7802	LDRB	r2, [r0, #0]
0x48E	7801	ADDS	r0, #1

SP = 0x2003FFF0, LR = 0x00000537, PC = 0x0000048C

Cambio PC y el LR, porque el BL primero guarda en LR la dirección de la instrucción inmediatamente siguiente al BL y luego carga en el PC la dirección de destino de la subrutina y como podemos ver el valor de PC si coincide con lo que se muestra en la ventana del Disassembly.

Pregunta 19

```
R0 0x0000055A
R1 0x20000004
```

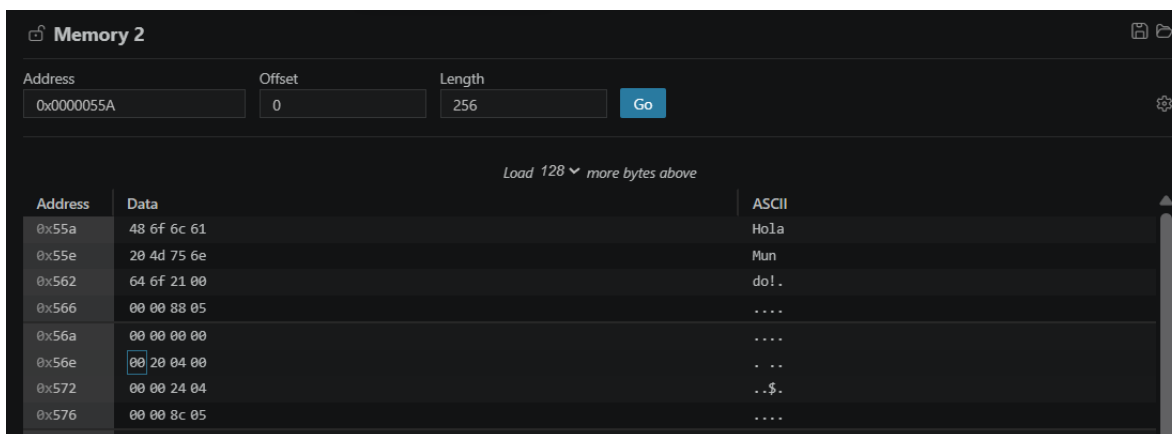
Los argumentos de my_strcpy pasan en los registros R0 y R1, donde R0 contiene el valor de src y R1 contiene el valor de dst, los valores son: R0 = 0X0000055A, R1 = 0X20000004

Pregunta 20

Memory 1	
Address	Offset
0x0000055A	0
Length	256
Go	
Load 128 ▾ more bytes above	
Address	Data
0x55a	48 6f 6c 61
0x55e	20 4d 75 6e
0x562	64 6f 21 00
0x566	00 00 88 05
0x56a	00 00 00 00
Load 128 ▾ more bytes below	

El contenido de la dirección src corresponde al String “Hola Mundo!” que esta almacenado en código ASCII

Pregunta 21

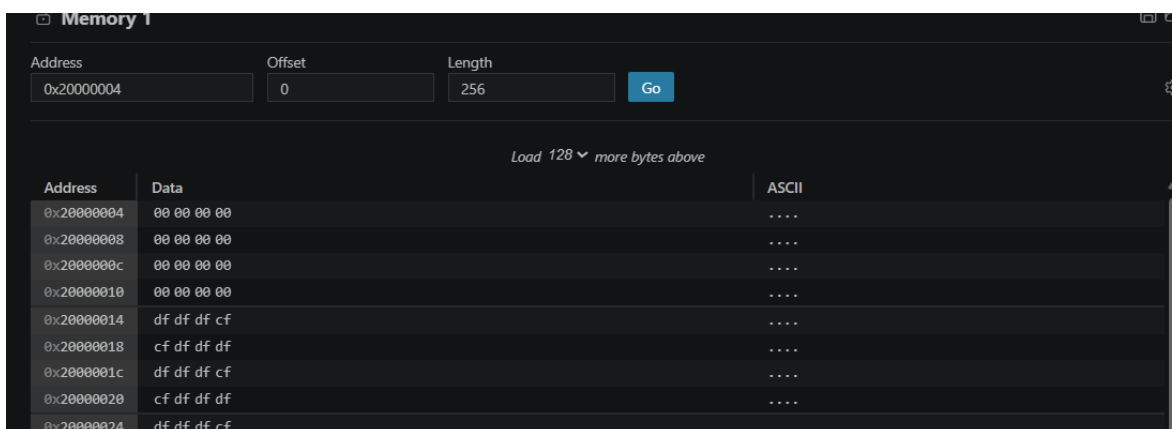


Memory 2

Address: 0x000055A Offset: 0 Length: 256 Go

Load 128 ▾ more bytes above

Address	Data	ASCII
0x55a	48 6f 6c 61	Hola
0x55e	20 4d 75 6e	Mun
0x562	64 6f 21 00	do!.
0x566	00 00 88 05	
0x56a	00 00 00 00	
0x56e	00 20 04 00	. . .
0x572	00 00 24 04	..\$.
0x576	00 00 8c 05	



Memory 1

Address: 0x20000004 Offset: 0 Length: 256 Go

Load 128 ▾ more bytes above

Address	Data	ASCII
0x20000004	00 00 00 00	
0x20000008	00 00 00 00	
0x2000000c	00 00 00 00	
0x20000010	00 00 00 00	
0x20000014	df df df cf	
0x20000018	cf df df df	
0x2000001c	df df df cf	
0x20000020	cf df df df	
0x20000024	df df df cf	

Como podemos ver confirmamos lo del paso 20 onde podemos ver que en el src tenemos “Hola Mundo!”, mientras que en dst tenemos caracteres vacíos porque my_strcpy no ha copiado nada aún

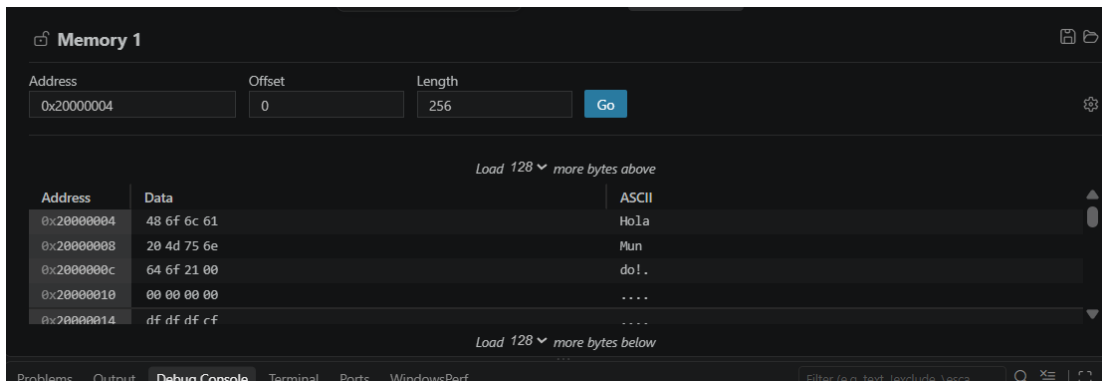
Pregunta 22

Como vimos anteriormente en src se almacena la cadena de texto “Hola Mundo!” junto al carácter nulo

Pregunta 23

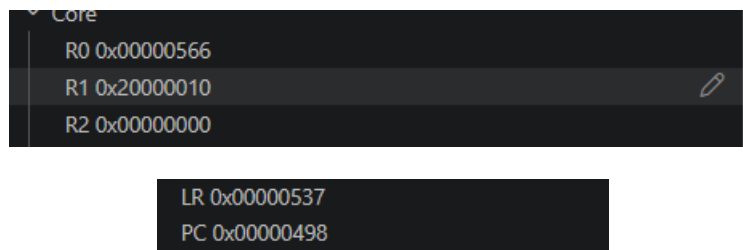
Como pudimos ver anteriormente el contenido inicial son puros ceros, debido a que aun no se ha ejecutado ninguna instrucción de my_strcpy que escriba datos en dst.

Pregunta 24



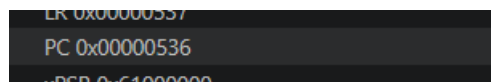
Se ejecuto la subrutina `my_strcpy` paso a paso observando el comportamiento del ciclo loop mostrado en la ventana de Disassembly, verificamos que en cada iteración del ciclo el carácter leído desde `src` se almacena en el registro `R2` mediante la instrucción `ldrb` y después se escribe en `dst` mediante la instrucción `strb`. Dicho ciclo lo repetimos 11 veces hasta que `R2` tomo el valor de `0x00000000`.

Pregunta 25



`src = 0x00000566`, `dst = 0x20000010`, `LR = 0x00000537`, `PC = 0x00000498`

pregunta 26



`PC = 0x00000536`

Pregunta 27

El valor del `pc` después de ejecutar la instrucción `bx` es igual al valor que tenía el `LR` antes de ejecutar dicha instrucción. Esto debido a que `bx lr` copia directamente el contenido del registro `LR` hacia el `PC`, lo que provoca que la ejecución regrese exactamente al punto donde se había guardado la dirección de retorno cuando se ejecutó el `BL my_strcpy` original.

Conclusiones:

- Se comprendió el procedimiento para poder integrar el código escrito en el lenguaje de C y ensamblador, logrando observar como una función desarrollada en ensamblador puede ser llamada directamente desde un programa de C mediante declaraciones externas correspondientes.
- Logramos comprender la función de los registros del procesador ARM Cortex-M4 durante la ejecución de un programa, principalmente de los registros para pasar argumentos, manejar la pila y almacenar la dirección de retorno de las funciones.
- Mediante el depurador y gracias también a la ventaja de Disassembly, observamos la relación entre el código escrito en C y las instrucciones de ensamblador que realmente ejecuta el procesador.
- Logramos observar el comportamiento de las rutinas `my_strcpy` y `my_capitalize`, de tal manera que logramos observar como la primera nos permite copiar una cadena de caracteres y la segunda modificar los caracteres de minúscula a mayúscula-

Bibliografía:

Arm Limited, "Directives Reference," *ARM Compiler armasm User Guide*. [Online]. Available: https://www.keil.com/support/man/docs/armasm/armasm_dom1361290000455.htm. [Accessed: Sep. 12, 2026].

Arm Limited, "Write Arm assembler functions," *Arm Learning Paths*. [Online]. Available: <https://learn.arm.com/learning-paths/embedded-and-microcontrollers/asm/>. [Accessed: Sep. 12, 2026]

Arm Limited, "KA003813," *Arm Support*. [Online]. Available: <https://support.arm.com/documentation/ka003813/1-0/>. [Accessed: Sep. 12, 2026]

Arm Limited, "Function parameters on 32-bit Arm," *Arm Community*. [Online]. Available: <https://developer.arm.com/community/arm-community-blogs/b/architectures-and-processors-blog/posts/function-parameters-on-32-bit-arm>. [Accessed: Sep. 12, 2026]

Arm Limited, "Arm Cortex-M3 and later: Basic integer math operations," *Arm Community*. [Online]. Available: <https://developer.arm.com/community/arm-community-blogs/b/architectures-and-processors-blog/posts/arm-cortex-m3-and-later-basic-integer-math-operations-32-bit-1-5>. [Accessed: Sep. 12, 2026]